

CMP-142 Object Oriented Programming

Lab-08

Topics:

- Inheritance
- Runtime Polymorphism
- File Handling

Instructions!

- This is a **graded lab**, you are strictly **NOT** allowed to discuss your solutions with your fellow colleagues, even not allowed to ask how he/she is doing, it may result in negative marking. You can **ONLY** discuss with your TAs or instructor.
- Indent your code properly.
- Use meaningful variable and function names. **Use the camelCase notation.**
- Save your work frequently. Make a habit of pressing CTRL+S after every line of code you write.
- **Do NOT use any global or static variable(s).** However, global named constants may be used.
- Make sure that there are **NO dangling pointers or memory leaks** in your programs.
- Make sure that all functions have been implemented and can be called in the main function. Otherwise, no marks will be awarded for that function.
- *Prototypes of some functions are omitted. You are supposed to complete them on your own. Moreover, there might be a case where some functions are ignored altogether while giving structure of class. You have to write and complete them.*

[Task 1] Business Intelligence System

[50 Marks]

Design and implement a Business Intelligence (BI) System for a multinational organization that utilizes advanced features of Inheritance, Runtime Polymorphism, and File Handling.

Also you need to Store all data persistently in files:

- Employee records
- Project details
- Department summaries

Other details are provided in class structures.

NOTE: In this task, you should logically think, which class can be inherited from which class. Each inheriting class must override the functions of the inherited class. Moreover, it is understood that you have to implement Constructor for each class.

Class Structures:

Class: Entity

This class represents the generic employee.

Attributes:

```
int id: Unique ID.  
std::string name: Name of the entity.
```

Public Member Functions:

```
virtual void displayDetails() const = 0: Display entity details.  
virtual void saveToFile(std::ofstream&) const = 0: Save entity data to a file.  
virtual void loadFromFile(std::ifstream&) = 0: Load entity data from a file.
```

Class: Employee

Attributes:

```
double baseSalary //Base salary of employee  
int skillLevel //Skill level (1-10)
```


Public Member Function:

Add `virtual double calculateSalary() const = 0`: Compute total salary dynamically

Class: **Manager**

Attributes:

`int teamSize`: Number of employees managed.

Public Member Functions:

Override `calculateSalary` to include bonuses based on team size in addition to their base salary.

Class: **Project**

Attribute:

`double budget`: Project budget.

`int deadlineDays`: Deadline in days.

`Employee** team`: Pointer to an array of employee pointers.

`int teamCount`: Number of employees in the team.

Also declare a private member function: `void expandCapacity()` which extends the capacity of the array by 2 times when it reaches its limits such that no previous data is lost and new one is added.

Public Member Functions:

```
void addEmployee(Employee* emp)
```

```
void removeEmployee(int empId)
```

```
double calculateProgress() const
```

 The function calculates the overall progress of a project based on the skills of the team. It looks at the salary of each team member (which we assume reflects their skill level), adds up everyone's skill, and then calculates the average skill of the team.

Class: Department

Attribute:

```
Project** projects:
```

 Pointer to an array of project pointers.

```
int projectCount:
```

 Number of projects in the department.

Also declare a private member function: `void expandCapacity()` which extends the capacity of the array by 2 times when it reaches its limits such that no previous data is lost and new one is added.

Member Functions:

```
double calculateEfficiency() const.
```

 // Method to calculate department efficiency based on project progress

```
void addProject(Project* project)
```

```
void removeProject(int projectId)
```

```
Save/load department data by overriding the base classes, including nested projects and employees working in the department
```

For calculating department efficiency, imagine you have a department with several projects. Each project is making progress (like 50%, 60%, etc.). This function adds up all the progress of the projects and then finds the average to give an overall idea of how well the department is performing.

For example: If one project is 80% done, and another is 60% done, the department's efficiency will be the average: $(80\% + 60\%) / 2 = 70\%$.

Depict the working of your inheritance (all functions must be used somewhere in the entire program) by dynamically creating the pointer to objects.

[Task 2] Polynomial Class **[30 Marks]**

Create a C++ class **Polynomial** that represents a polynomial and implements key operations using object-oriented programming concepts such as inheritance, virtual functions, and runtime polymorphism.

The polynomial should be represented by its coefficients and powers. For example, the polynomial $3x^3 + 2x^2 + 5x + 1$ will be represented by an array of coefficients $\{1, 5, 2, 3\}$ corresponding to the terms $1 + 5x + 2x^2 + 3x^3$.

Base Class: Polynomial

Protected Members:

<code>int degree;</code>	<code>// Degree of the polynomial</code>
<code>double* coefficients;</code>	<code>// Dynamically allocated array for coefficients</code>

Public Member Functions:

<code>Default Constructor</code>	
<code>Parameterized Constructor</code>	
<code>Copy Constructor</code>	
<code>Assignment Operator</code>	
<code>Destructor;</code>	<code>//virtual destructor</code>
<code>virtual double evaluate(double x) const</code>	<code>// Evaluate the polynomial at a given point x</code>
<code>virtual void display() const</code>	<code>// Display the polynomial in standard form</code>
<code>virtual Polynomial derivative(int x)</code>	<code>//Take derivative of the polynomial up to x.</code>
<code>virtual int calculateDerivative(int) = 0;</code>	<code>//Return the result after putting the value in the</code>
<code>calculated derivative. This function should be called from main.</code>	

Derived Class: **MonicPolynomial**:

A polynomial is called a monic polynomial where the leading coefficient is always 1.
You can implement this class by inheriting the previously implemented Polynomial class
(Deduce the type of inheritance by yourself).

Public Member Functions:

Constructor
Override the virtual functions of base class.

Derived Class: **SparsePolynomial**

A **Sparse Polynomial** is a polynomial that efficiently stores only non-zero terms to save space. It uses dynamic memory but avoids storing zero coefficients. You can implement it by inheriting the Polynomial class (deduce the type yourself).

Private Members:

```
int termCount;           // Number of non-zero terms
int* powers;             // Dynamic array for storing exponents
double* coeffs;          // Dynamic array for storing non-zero coefficients
```

Public Member Functions:

Constructor
Copy Constructor
Assignment Operator

~SparsePolynomial() override;
Override all the virtual functions of base class.

In the main function, show the working and correctness of your implemented classes by displaying the working of each and every function implemented.